

Reviewer Pack — Minimal Index of Algebraic Quotients and DPLL Proof Depth Co...

Entry 786703f89730 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 00:00:57 UTC

Minimal Index of Algebraic Quotients and DPLL Proof Depth Correlation

Entry ID: 786703f89730 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 00:00:57 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Quotient Theory) **Field B** (complexity object): DPLL Search Trees

Statement:

For every Boolean CNF φ with m clauses, the minimal index of algebraic quotients in the quotient group of its associated symmetric matrix is linearly correlated with its DPLL proof depth $d(\varphi)$, such that $\log(\text{min_index}) = \Theta(d(\varphi))$.

Rationale (proposer’s reasoning):

Algebraic quotient theory provides a rich framework for studying group actions on algebraic structures. In the context of Boolean CNFs, analyzing the quotient structure of symmetric matrices might reveal hidden symmetries that are related to the complexity of DPLL proof search trees.

Taxonomy category: algebraic_quotient_theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b13464df3029480d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For every Boolean CNF φ with m clauses, if $\log(\text{min_index})$ falls within ± 1 of $d(\varphi)$, and the correlation coefficient (r) is greater than or equal to 0.7 over 30 random seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal index of algebraic quotients" AND "DPLL proof depth" AND algebraic geometry - "quotient group symmetric matrix" AND DPLL search trees AND CNF Boolean - "algebraic geometry" AND "DPLL algorithm" AND correlation minimal index

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.5s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_random_cnf(n: int, m: int) -> list:
    cnf = []
    for _ in range(m):
        clause = [random.randint(1, n), random.randint(-n, -1)]
        cnf.append(clause)
    return cnf

def construct_symmetric_matrix(cnf: list, n: int) -> list:
    matrix = [[0] * (2 * n) for _ in range(2 * n)]
    for clause in cnf:
        i, j = abs(clause[0]) - 1, abs(clause[1]) - 1
        matrix[i][j + n] = 1
        matrix[j + n][i] = 1
    return matrix

def gaussian_elimination(matrix: list) -> list:
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        # Find a non-zero pivot in the current column
        pivot_row = next((r for r in range(i, rows) if matrix[r][i] != 0), None)
        if pivot_row is None:
            continue
        # Swap rows to move the pivot to the diagonal
        matrix[i], matrix[pivot_row] = matrix[pivot_row], matrix[i]
        # Eliminate non-zero entries below the pivot
        for r in range(i + 1, rows):
            factor = Fraction(matrix[r][i], matrix[i][i])
            for c in range(cols):
                matrix[r][c] -= factor * matrix[i][c]
    return matrix
```

```

def min_index_of_quotients(matrix: list) -> int:
    reduced_matrix = gaussian_elimination(matrix)
    rank = sum(1 for row in reduced_matrix if any(row))
    return rank

def dpll_proof_depth(cnf: list, n: int) -> int:
    def dpll(clause_set, assignment):
        if not clause_set:
            return 0
        unit_clauses = [c for c in clause_set if len(c) == 1]
        if unit_clauses:
            literal = unit_clauses[0][0]
            new_assignment = assignment.copy()
            new_assignment[literal] = True
            return dpll([c for c in clause_set if literal not in c], new_assignment)
        pure_literals = [l for l in range(1, n + 1) if (l not in assignment and -l not in assignment)]
        if not pure_literals:
            return float('inf')
        literal = pure_literals[0]
        new_assignment = assignment.copy()
        new_assignment[literal] = True
        return dpll([c for c in clause_set if literal not in c], new_assignment)
    return dpll(cnf, {})

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    min_indices = []
    proof_depths = []

    for n in n_values:
        cnf = generate_random_cnf(n, n * (n - 1) // 2)
        matrix = construct_symmetric_matrix(cnf, n)
        min_index = min_index_of_quotients(matrix)
        proof_depth = dpll_proof_depth(cnf, n)

        if min_index == 0 or proof_depth == float('inf'):
            return {
                "metric_name": "log_min_index",
                "metric_value": None,
                "instances_tested": len(n_values),
                "n_max": n,
                "conjecture_holds": False,
                "counterexample": "min_index_zero_or_infinite_proof_depth"
            }

        min_indices.append(math.log(min_index))
        proof_depths.append(proof_depth)

    correlation_coefficient = sum((x - mean_x) * (y - mean_y) for x, y in zip(min_indices, proof_depths)) / (
        math.sqrt(sum((x - mean_x)**2 for x in min_indices) * sum((y - mean_y)**2 for y in proof_depths))
    )
    mean_log_min_index = sum(min_indices) / len(min_indices)

    return {
        "metric_name": "log_min_index",

```

```

        "metric_value": mean_log_min_index,
        "instances_tested": len(n_values),
        "n_max": max(n_values),
        "conjecture_holds": abs(correlation_coefficient) >= 0.7,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_log_min_index = sum(r["metric_value"] for r in results if r["metric_value"] is not None)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_log_min_index} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_log_min_index} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient_not_met\" first_failing={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6071d525.py", line 117, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6071d525.py", line 98, in run_trial
    correlation_coefficient = sum((x - mean_x) * (y - mean_y) for x, y in zip(min_indices, proof_depths))
                                ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6071d525.py", line 98, in <genexpr>
    correlation_coefficient = sum((x - mean_x) * (y - mean_y) for x, y in zip(min_indices, proof_depths))
                                ~~~~~^
NameError: name 'mean_x' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture’s conditions. | next: Review and debug the test code to ensure it can run to completion without errors.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	27790
2	preregistration	ollama_remote	glm4:latest	0	0	12287
3	novelty	ollama_remote	glm4:latest	0	0	8422
4	novelty	ollama_remote	glm4:latest	0	0	13328
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23852
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23918
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26109
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15184
9	judge	ollama_remote	glm4:latest	0	0	22959

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 173849 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/786703f89730.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/786703f89730.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/786703f89730.tar.gz (if generated)